

ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

Slice & Spice Pizzéria

2025.

Szakma:
SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ
5 0613 12 03

Készítették:
Sándor-Spanyol Marcell
Plézer Zsombor
Feil Bence

Tartalomjegyzék

I.	Bevezetés	6
II.	Témaválasztás indoklása	6
III.	Fejlesztői dokumentáció, technikai leírás	7
1.	Specifikáció	7
2.	Alkalmazott fejlesztői eszközök	8
3.	Adatmodell.....	9
4.	Algoritmusok	13
5.	Tesztelési Dokumentáció.....	18
IV.	Felhasználói dokumentáció.....	27
1.	A program általános specifikációja, célja	27
2.	Rendszer követelmények	28
3.	A program telepítése	29
4.	A program használatának részletes leírása	31
V.	Összegzés.....	36
VI.	Ábra jegyzék	36
VII.	Irodalomjegyzés, hivatkozási jegyzék.....	39

Nyilatkozat

Alulírott Feil Bence büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 18.

Feil Bence Zoltán

Név

Nyilatkozat

Alulírott Sándor-Spanyol Marcell büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 18.

Sándor-Spanyol Marcell

Név

Nyilatkozat

Alulírott Plézer Zsombor büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 18.

Plézer Zsombor

Név

I. Bevezetés

A pizza az egyik legnépszerűbb étel a világon, amely nemcsak Olaszországban, hanem számos országban meghatározó szerepet kapott az étkezési szokásokban és hihetetlen mély és gazdag kultúrával rendelkezik. A pizzériák az étkezési élményeket és a közösségi találkozókat egyaránt szolgálják, így a pizza nem csupán egy étkezési lehetőség, hanem egy társasági esemény is. Téma választásunk célja, hogy az éttermek és vendéglátóhelyek működését bemutassuk, különös figyelmet szentelve a pizza iparág egyre növekvő népszerűségének és az iparági trendeknek.

II. Témaválasztás indoklása

Az éttermek működése iránti érdeklődésünk hosszú ideje megvan, mivel a vendéglátás és az éttermek világa rendkívül dinamikus és folyamatosan fejlődik. Az elmúlt években a pizza, mint gyors étkezési lehetőség, de egyben prémium minőségű étel is, reneszánszát éli, egyre több helyen kerülnek előtérbe különleges, helyi alapanyagokkal készített változatok, vagy éppen a gyors, kényelmes kiszolgálásra építő éttermek. Az aktuális trendek és a vendégek igényei alapján az iparág folyamatos innovációt kíván, így úgy gondoltuk, hogy a pizzériák működése és jövője fontos és releváns téma. A témához kapcsolódó kutatásokat végeztünk, amelyek rávilágítottak az ipar különböző kihívásaira és lehetőségeire, a fenntarthatóságra és az éttermi szolgáltatás minőségére.

Az éttermek a gyakorlatban való működésüket és az új szolgáltatási modelleket, mint például a házhoz szállítást és a mobilalkalmazások használatát, észben tartva fejlesztettük weboldalunk. Különböző pizzériák látogatása is hozzájárult a témával kapcsolatos tapasztalataink bővítéséhez. Hiszen kik jobbak tudnának belátást nyújtani az ipar belső működésébe, mint a benne dolgozó, lelkes munkás emberek, akik az életüket szentelték a mesterségnek. Mindezek segítettek abban, hogy átfogó képet alkothassunk egy pizzéria napi működéséről, kihívásairól és lehetőségeiről.

III. Fejlesztői dokumentáció, technikai leírás

1. Specifikáció

Egy, a lehetséges vásárlókkal folyamatos kommunikációban álló weboldalnak számtalan funkcióra és egységes formátumra van szüksége a gördülékeny és probléma mentes vásárlási élmény nyújtásához, ezt a gondolatot a szívünkön hordozva fejlesztettük weboldalunk.

Felhasználó kezelés: Biztonsági és kényelmi szempontok miatt az érvényes rendelés leadásához a felhasználóknak egy vásárlói fiókot kell létrehozniuk (ha már rendelkeznek aktív fiókkal akkor egyből bejelentkezhetnek) és bejelentkezniük. Így a fiókjukon keresztül tudják követni és felgyorsítani a jövőbeli rendelésüket.

Rendelés: A rendelés folyamatának első lépése az adott termékre való kattintás, ami egy virtuális kosárhoz lesz hozzá adva, ami eltárolja azt, ezután a vásárlók dönthetnek úgy, hogy egyből a kasszához „állnak” és megrendelik a kosarukban lévő termékeket, vagy folytatják nézelődést és később fejezik be a rendelést. A rendelés folyamán a szükséges adatok pontos megadására való kötelezés limitálja a szállítási hibákat.

Rendelés követés: A weboldalon a rendeléseim fülön lehetőség van az oldal látogatóknak a múltbéli rendelések megtekintése, és az aktív rendelések követésére. Ebbe beletartozik a rendeléskor leadott összes adat (név, pizzák száma és típusa, fizetési mód, státusz, szállítási cím, fizetés módja és fizetendő összeg) ellenőrzésére.

Jogosultság kezelés: Az oldalon lehetőség van különböző jogosultságú fiókok használatára (a megfelelő hitelesítés után természetesen). Továbbá a tulajoknak is lehet biztosítani admin fiókot, amelyen keresztül tudnak új pizzákat létre hozni, meglévőket változtatni és hozzá férnek a rendelési előzményekhez.

2. Alkalmazott fejlesztői eszközök

C#: A weboldal „backend” részét, azt a részt, amelyet a felhasználók nem látnak úgymond a függöny mögötti varázslatot C Sharp nyelven lett megírva, a Visual Studio 2022-vel, .net (8.0) környezetben.

React: A frontend részt, azt a felületet, amellyel a felhasználók közvetlen kapcsolatban állnak React felhasználásával, amely egy ingyenes, nyílt forráskódú JS könyvtár mellyel, a Visual Stúdió környezetében fejlesztettük.

JavaScript: A JS a modern világháló egyik alap eleme és gyakori nyelve. A mi projektünkben az adatbázissal való megbízható és hibamentes kommunikációért felel. Adatokat kér le az adatbázistól, adja őket tovább, hoz létre újakat, módosítja esetleg kitörli azokat.

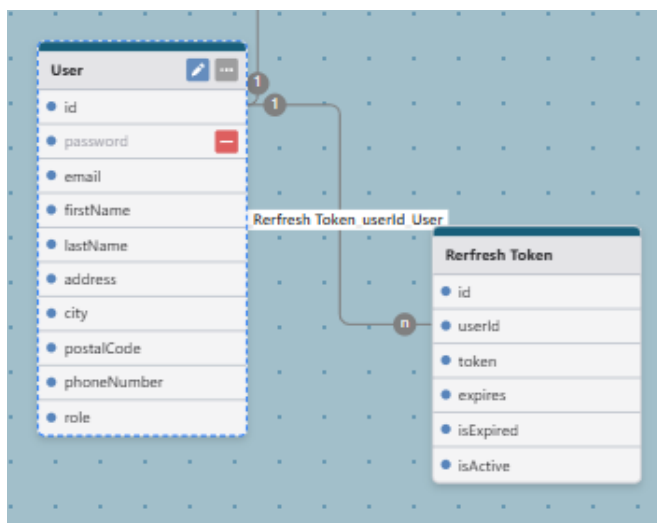
HTML: JS - hez hasonlóan a HTML is a World Wide Web egyik elengedhetetlen építő eleme. Feladata, hogy megfelelő környezetet biztosítson a beérkező adatoknak és megjelenítse azokat.

CSS: A CSS HTML, JS triumvirátus harmadik tagja, az esztétikus dizájn biztosításában van szerepe, amelyhez különböző könyvtárak állnak rendelkezésre.

Vite: Egy frontend fejlesztői eszköz, ami a Node.Js alatt fut. Francia szó, jelentése gyors és ez a fő szempontja a használata mellet való döntésünket.

SQLite: Az sqlite egy SQL alapú, ingyenes és nyílt-forrás kódú relációs adatbázis C nyelven írva, amely arra szolgál, hogy biztonságosan tárolja az adatokat.

3. Adatmodell



2. Ábra User szekció

A User szekció

Az adatmodell központi eleme a **user** tábla, itt tároljuk el a felhasználók adatait, és a hozzátartozó Refresh Token tábla adatait mely a beléptetéshez szükséges.

Id: egy egyedi 36 karakterből álló láncot ad neki.

Továbbá ez az azonosító fogja biztosítani a kapcsolatot a többi táblával.

A *password*-öt, *email*-t, és a *phone number*t a visual studio képes saját maga kezelni az ASP.NET Core Identity-n keresztül így azokkal különösebben nem kell foglalkoznunk.

A *firstName* a felhasználó család nevét tárolja el, egyik alkotója az oldal hivatkozásának a felhasználóra, string típusú.

A *lastName* a felhasználó keresztnéve, másik alkotója az oldal hivatkozásának a felhasználóra, string típusú.

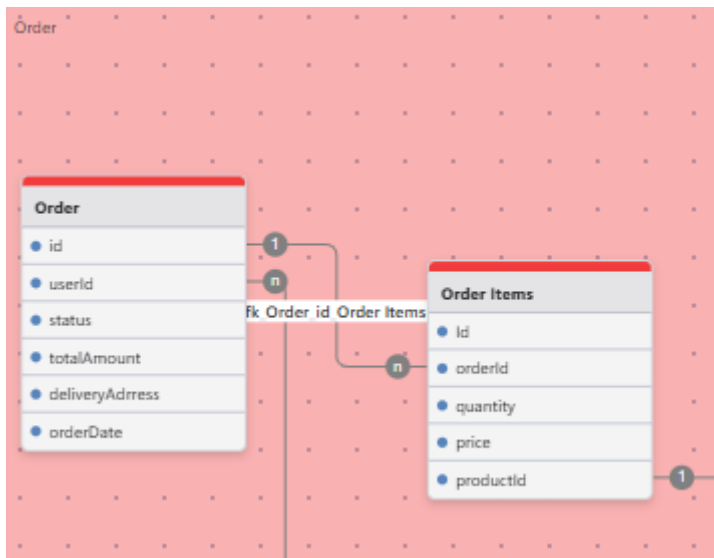
Az *address* a felhasználó lakcímét tárolja melyet majd a rendelésnél fog felhasználni a weboldal, string típusú

A postal code a város irányító számát fogja eltárolni, string típusú.

A role az a felhasználó szerepkörét takarja, ami lehet admin, vagy user.

Refresh Token

A legnagyobb biztonság érdekében weboldalunk JWT védelmet alkalmaz refresh tokenekkel. Ez azt takarja, hogy amikor a felhasználó belép az oldalra kap egy 30 percre elegendő hitelességi „kártyát” és egy frissítési kártyát. Amikor lejár a hitelességi kártya a rendszer megnézi, hogy rendelkezik-e a felhasználó aktív frissítési kártyával és az újfenti belépés nélkül ad neki egy újabb hitelességi kártyát.



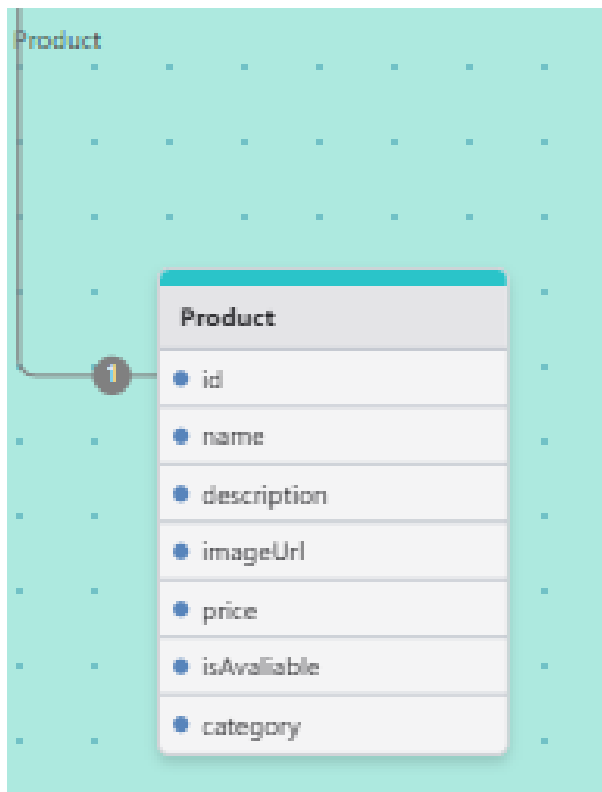
3. ábra Order szekció

Az **Order** rendszer

Amikor a vásárló rákattint a rendelés gombra akkor egy végponton keresztül átadjuk a virtuális **Cart**, és a **Cart-ítemek** tartalmát az **Order** és az **Order Items** táblákba. Itt a két tábla között egy- a többhöz kapcsolat áll fenn hiszen egy rendelésben több rendelési tétel lehet. Továbbá az **Order** tábla rendelkezik két fontos

tulajdonsággal, amivel a **Cart** nem. Az első a *statusz*: a rendelés jelenlegi állapotát jelzi valós időben, amely lehet, pending, completed, és cancelled. A második pedig a *deliveryAddress* mely a kiszállítási címet tárolja. A weboldal alapértelmezetten felfogja ajánlani a regisztrációnál megadott címet, de természetesen a rendelő is megadhatja magától, ha lakhelyétől eltérő címre szeretne rendelni weboldalunkról.

Az **Order Items** tábla itt egy-az egyhez kapcsolatban kapcsolódik a **Products** táblához a *productId* és a **Products** tábla saját *id*-ján keresztül



4. ábra Product szekció

A Products Tábla

A Products táblában tároljuk el az oldalon megvásárolható termékeket és az adataikat ezek a következők:

Id: Minden egyes termék saját, egyedi szám alapú azonosítója, ezen keresztül kapcsolódik egy-a többhöz, módon a virtuális **Cart Items** és az **Order Items**-hez.

name: A termék megnevezése, string típusú.

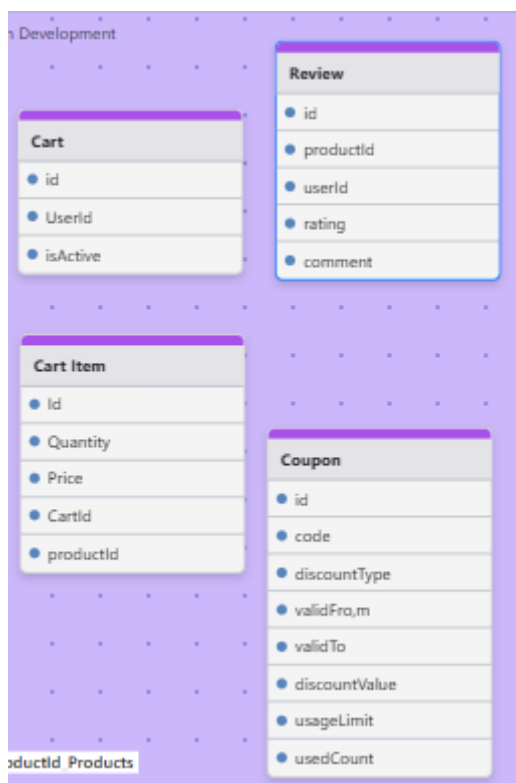
description: A termék rövid (Max 250 karakter hosszú) leírása.

ImageUrl: A termékhez tartozó kép url-jét tárolja el. Ennek segítségével tudunk képet társítani a termékhez a weboldalon.

price: A termék ára, forintban értetendő mely tartalmazza az ÁFÁ-t, decimál típusú, hiszen a jövőben a kedvezmények és akciókon keresztül lehetséges, hogy nem kerek egész szám lesz az új ár.

isAvaliable: egy boolean (igaz v hamis) változó típusú tulajdonság mely arra szolgál, hogy a termék elérhetőségét jelezze (igaz, ha van készleten, hamis a kifogyott vagy nem tudjuk biztosítani).

category: A category tulajdonság egy Enum (Enumerátor) típusú tulajdonság, amely azt jelenti, hogy ez a tulajdonság csak is az előre megadott értékkel rendelkezhet, jelen esetben Drink vagy Pizza, ez adja meg hogy a termék ital vagy étel-e.



5. ábra In Development szekció

In Development szekció

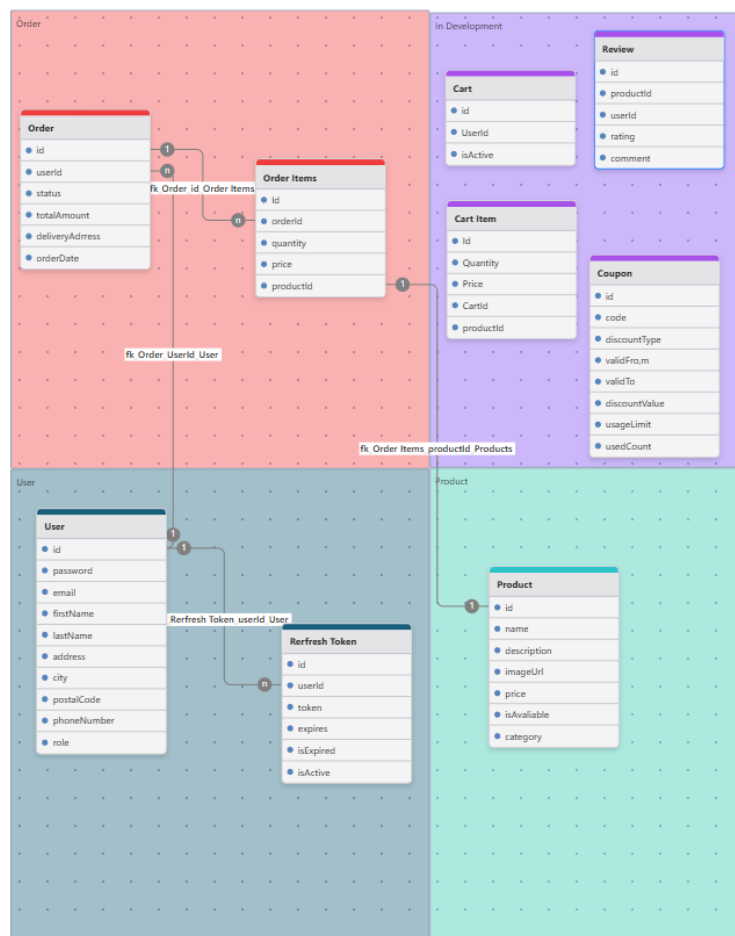
Az In Development szekció azokat a lehetséges funkciókat tartalmazza melyekkel a weboldal bővíthető lehetne a jövőben. A mi esetünkben ezek a **Review**, **Coupon**, **Cart** és a hozzátartozó **Cart Items** táblában valósulhat meg.

A **Review** tábla lehetővé teszi a felhasználóknak, hogy értékeljék a terméket, illetve, hogy elmondhassák a gondolataikat és tapasztalataikat. A tábla a `userId` keresztül kapcsolódna a user táblához egy a többhöz kapcsolatban, egy userhez több review is kapcsolódhatna.

A **Coupon** tábla a lehetséges akciók és kedvezmények kezelésére szolgálna

megadva a kezdemény típusát, validatásának idejét, a kedvezmény mértékét, hányszor lehet összesen felhasználni és eddig hányan használták.

Cart és a **Cart Item** tábla jelenleg is létezik, de a többi táblától eltérően (nem számítva az in development szekcióban lévő táblákat) a local Storage-ben van eltárolva és a frontenden belül a képen látható tulajdonságokkal rendelkezik, viszont nincsen definiált táblája a backendben. Ha a backendben is lenne saját táblája lehetséges lenne, hogy a korábbi kosaraknak az adatait visszakerjünk, amikből később értékes kutatási adatokat lehetne kinyerni, mint például melyik terméket törölték ki legtöbbször a vásárlók. A **Cart** és a **Cart Item** tábla közötti kapcsolatot úgy lehetne elképzelni mintha a kosáron belül minden egyes termék a saját külön kis szatyrában lenne, aminek meg van a saját kis azonosítója. A kapcsolat a kosár és a „szatyrok” között, logikusan, egy a többhöz. Ami azt jelenti, hogy egy kosárban több kis szatyor lehet. Viszont a specifikus szatyrok csak egy kosárhoz tartozhatnak. Az `IsActive` boolean (igaz vagy hamis) változónak fontos szerepe van. Mégpedig, hogy amikor egy terméket hozzá akarunk adni a kosárhoz akkor ez alapján dönti el, hogy létre kell-e hoznia egy új kosarat mert még nincs vagy már van kosárunk és csak abba kel bele raknia.



6. ábra Teljes adatbázis modell

4. Algoritmusok

1. A belépés és a regisztráció folyamata

A regisztrációt nagyrészt a Register.tsx fájlban kezeljük le. Itt először importáljuk a felhasználni kívánt külső forrásokat, mint például a react esemény és állapot kezelő, a szerver címe, és a rendszer üzenetek designért felelős toasty-t. Itt még felvesszük a jelszó és az email regexeket amelyek ellenőrizni fogják, hogy megfelelő formában vannak-e.

Utána létrehozunk egy interfészt, ez fogja először fogadni a felhasználókat amikor a regisztráció gombra kattintanak. Ezt az interfészt miután kitöltik mi eltároljuk egy useState-ben. Természetes nem hihetünk vakon felhasználóknak ezért le kell ellenőriznünk, hogy helyesen adták meg az adatokat, és csakis akkor, ha sikeres eredménnyel térnek vissza az ellenőrzéseink engedhetjük őket tovább.

A sikeres ellenőrzési csekk után felkészíthetjük az adatokat a szerver iránti tovább küldésre majd el is küldjük, ahol a backend sikeresen fel tudja dolgozni a már biztosan helyes regisztrációs adatokat.

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  const v1 = EMAIL_REGEX.test(formData.Email);
  const v2 = PWD_REGEX.test(formData.Password);

  if (!v1 || !v2) {
    setErrMsg("Invalid Entry");
    return;
  }

  const requestData = {
    Email: formData.Email,
    Password: formData.Password,
    FirstName: formData.FirstName,
    LastName: formData.LastName,
    Address: formData.Address,
    City: formData.City,
    PostalCode: formData.PostalCode,
    PhoneNumber: formData.PhoneNumber,
  };
};
```

7. ábra Regisztrációs adatok előkészítése

```
const response = await fetch(`${BASE_URL}/api/account/register`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(requestData),
});

if (!response.ok) {
  const errorText = await response.text();
  const errorData = JSON.parse(errorText);

  if (Array.isArray(errorData) && errorData.length > 0) {
    const details = errorData.map(e => e.description || e).join(", ");
    setErrMsg(`Registration error: ${details}`);
  } else {
    setErrMsg("Registration failed");
  }
  return;
}

toast.success("Registration successful!");
setSuccess(true);
```

8. ábra Regisztrációs adatok elküldése a végpontra

A response konstans először megkeresi a végpontot az URL-je alapján majd megmondja, hogy mit szeretne kezdeni ezzel a „csomaggal”, amit elfog neki küldeni. Ez a method ami egy POST jelen esetben ez azt jelenti, hogy szeretné hozzá adni az adott végponthoz a bodyban megadott csomagot és fel is készíti erre a csomagra. Ez a csomag a mi helyzetünkben a felhasználó által megadott adatok a megfelelő formában. A hiba kezelés az If (Array elágazással van lekezelve.

2. A bejelentkezési folyamat

A regisztrációhoz hasonlóan itt is az importokkal kezdjük. Név szerint:

setAuth()–Beállítja a szükséges hozzáférési státusztuseAuth()– egy speciális hook ami hozzáférést biztosít az általunk a useAuth.ts fájlban definiált hozzáférési állapotokhoz és a SetAuth()-hoz

useNavigate()– Az oldalon való navigáláshoz használjuk fel amikor átirányítjuk a felhasználót a sikeres bejelentkezés után.

useLocation– Lehetővé tesz a felhasználó jelenlegi helyének való lekérdezését

Természetesen ezeket el is tároljuk azonnal konstansokként, hogy később gond nélkül használhassuk őket, amikor szükségünk van rájuk.

A login felület dizájnját a hozzá tartozó login.jsx fájlban írtuk meg, itt kérjük be a jelszót és az email címet, és tároljuk el konstansokban a. tsx fájlban. Ezután következhet a belépési folyamat logikája.

```
const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();

  try {
    const response = await fetch(`${BASE_URL}/auth`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "include",
      body: JSON.stringify({ email, password }),
    });

    if (!response.ok) {
      const message = await response.text();
      throw new Error(message);
    }

    const data = await response.json();
    const decoded = parseJwt(data.accessToken);
```

9. ábra Bejelentkezési adatok elküldése

A handleLogin egy aszinkron művelet, mely a regisztrációhoz hasonlóan először egy fetch-el megkeresi a végpontját, amihez elszívesztene küldeni a belépési kérelmeket. Itt is POST lesz a kérésünk típusa és a header is megfog egyezni a regisztrációjához, viszont itt hozzáadjuk a credentialst ami megengedi, hogy majd utólag rakjunk még ehhez a csomaghoz bizonyos dolgokat, a mi esetünkben egy JWT token, amiről hamarosan fogok részletesebben is beszélni. Az if(response.ok) elágazás kezeli le az esetleges hibákat. Ha sikeresen el tudtuk küldeni az adatokat és a végpont validálta őket és visszaküldött egy token, amit mi dekódolunk. és v, és csakis akkor, lekéri a Jwt token és dekódolja azt.

```
navigate(from, { replace: true });
} catch (err: any) {
  setErrorMessage(err.message || "Login failed");
}
```

10. ábra A bejelentkezés utáni műveletek

```
setAuth({
  email: decoded.UserInfo.email,
  roles: decoded.UserInfo.roles,
  accessToken: data.accessToken,
});
```

11. ábra Feltöltjük a setAuthot a kinyert adatokkal

Ha nincsen semmi hiba akkor visszaküldjük a felhasználót a főoldalra, ha viszont igen akkor kiírjuk a hibát. És legvégül a legfelül beimportált setAuth()-ba feltöltjük a belépési adatokat és a dekódolt jwt token, ami tartalmazza a validált belépési adatokat és a hitelességi tokenünket.

3.Axios Token kezelés

```
const responseIntercept = axiosPrivate.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const prevRequest = error?.config as InternalAxiosRequestConfig & {
      sent?: boolean;
    };

    const isExpiredToken =
      error?.response?.status === 401 ||
      (error?.response?.data as any)?.message
        ?.toLowerCase()
        ?.includes("expired");

    if (isExpiredToken && !prevRequest?.sent) {
      console.log("Token appears to be expired, attempting refresh");
      prevRequest.sent = true;

      try {
        const newAccessToken = await refresh();
        if (prevRequest.headers) {
          prevRequest.headers["Authorization"] = `Bearer ${newAccessToken}`;
        }
        return axiosPrivate(prevRequest);
      } catch (refreshError) {}
      console.error(
        "Failed to refresh token during request:",
        refreshError
      );

      throw error;
    }
  }
);

return Promise.reject(error);
};
```

12. ábra A kötelező header hozzáadása az összes kéréshez

Az Axios rendszer biztosítja a weboldalunkon a zökkenőmentes és biztonságos használatot mivel a jwt token önmagában nem elég a teljesen biztonságos böngészéshez. Az oldalon egészen addig lehet, böngészni, amíg van érvényes használati tokenünk. Viszont, ha esetleg feltörik a fiókunkat hogyan tudjuk megállítani a már validált tokennel rendelkező illetéktelen betörőt. Úgy, hogy 30 percként új tokenel kell rendelkeznie minden aktív felhasználónak különben a kérései el lesznek utasítva. De nem lenne realiztikus 30 percenként kijelentkeztetni a felhasználókat. Erre a problémára kínál megoldást az axios. Kezdsnek beimportáljuk a szükséges külsőségeket. A már ismert useAuth-ot a useRefreshToken-t amely a lejárt token esetén való frissítésért felelős, illetve a useAxiosPrivate-ot, és eltároljuk őket konstansokként.

A responseIntercept-nek két nagyon fontos feladata van. Először is, hogy az összes ki és bejövő kéréshez hozzá rakjon egy headert amely arra fogja kötelezni a kérést, hogy rendelkezzen egy validált tokenel, ellentétes esetben utasítsa el.

Másodjára pedig, hogy ha a 401-as hibába (validálatlan token) ütközünk próbálja meg frissíteni a tokenünket a useAxiosPrivate()-n keresztül a useRefreshToken() algoritmussal és még egyszer próbálja meg elvégezni a kérést.

4. A token frissítés algoritmus

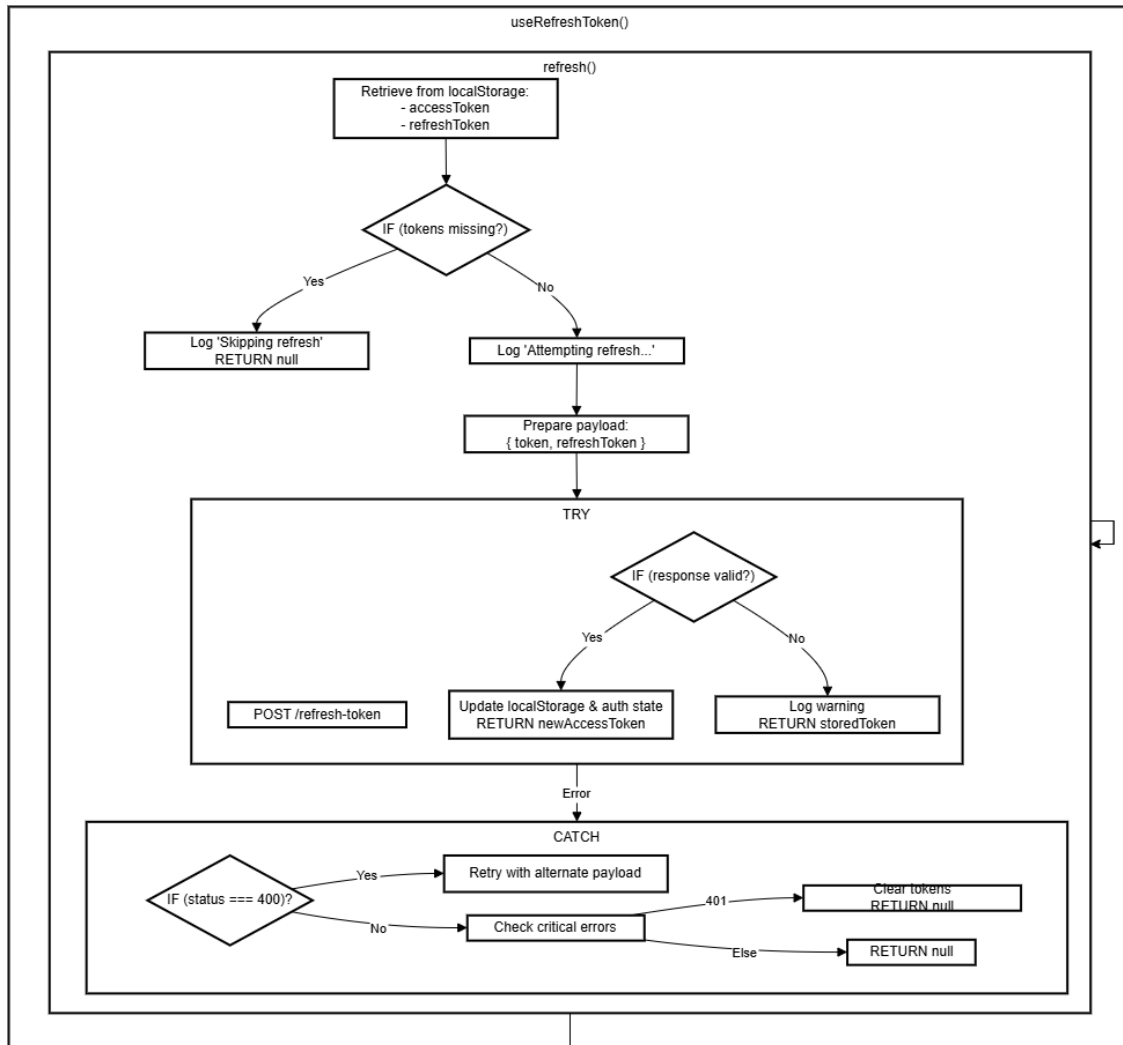
Három részre osztható a useRefreshToken.tsx, az első a már megszokott importálás, a második a AuthState interfész definiálása, és legvégül a frissítési algoritmust, amelyet a useAxiosPrivate meghív.

Az első részben a jwt-t vissza kódoló a parseJWT-t, az axiost magát, és a configot importáljuk be.

A második részben megmondjuk, hogy az AuthState interfész rendelkezzen hozzáférési tokennel (accessToken), a frissítő tokennel (refreshToken) a felhasználó szerepkörével (role) és a hitelesítési változóval (isAuthenticated) melyek mindegyike kötelezően megkell, hogy legyen.

A harmadikrészben megírt algoritmust vegyük szemügyre lépésenként.

1. lépés: Megszerzi a tokeneket amelyeket a localStorage-ben tárolunk el.
2. lépés: Leellenőrzi, hogy mindkét token megvan-e, ha akár csak az egyik is hiányzik, azonnal hibára fut.
3. lépés: Egy requestPayload nevű konstansban eltárolja a lekért tokeneket.
4. lépés: Elküldi a frissítési kérelmet a refresh-token végpontnak
5. lépés: A beimportált ParseJwt-vel dekódolja a visszakapott jwt csomagot.
6. lépés: A dekódolt JWT csomagból vegye ki az AuthState-nek szükséges adatokat., illetve a lejáratí időt.
7. lépés. Le frissíti az authState()-et és vissza tölti a local storage-be.
8. lépés: Ha a backend véletlen visszautasítja rossz formátum miatt újra próbálja egy alternatív elrendezéssel
9. lépés: Ha még így is sikertelen volt a token frissítés és továbbra is 401-as hibát kap vissza akkor elutasítja a jelenlegi és az összes jövőbeli kérését a felhasználónak és többször nem próbálja újra.



13. Ábra useRefresh Struktogram

5. Tesztelési Dokumentáció

1. Első Tesztet: Jelszó változtatás:

Elvárás: A felhasználó sikeresen megtudja változtatni a jelszavát az erre kijelölt oldalon a program utasításait pontosan követve.

Művelet: Az első mezőbe az oldal kérésének megadjuk a régi jelszót. Következőnek megadjuk az új jelszót. Végezetül megerősítésként még egyszer beírjuk az új, beállítani kívánt jelszót.

Várt eredmény: A program ellenőrizni fogja, hogy a két jelszó megegyezik és amannyiban ez a vizsgálat sikeres rá kattinthatunk a „Jelszó megváltoztatása” gombra, melynek sikerességéről az oldal értesíteni fogj a felhasználót.

Ellenkező esetben értesítse a felhasználót a fennálló problémáról és ne engedje megváltoztatni a jelszavát.

Valós eredmény: Megegyezik a várt eredménnyel, abban az esetben, ha a jelszók nem egyeznének a program egyöntetűen, univerzális piros ikonnal jelzi az adat mező hibásságát és szöveges értesítést is ad róla. Aktív állapotban (a felhasználó a mezőbe kattint) emlékezteti a jelszavak érintő követelményekről.



17. ábra Sikeres jelszó változtatás üzenet

Failed to update password. Please try again.

14. ábra Sikertelen jelszó változtatás üzenet

Must match the new password input field.

15. ábra Nem egyező jelszavak hiba üzenet

12 to 24 characters. Must include uppercase and lowercase letters, a number and a special character. Allowed special characters: ! @ # \$ % _

16. ábra Jelszó követelmények üzenet

2. Második Teszteset: A végpontok tesztelése

A weboldalunk végpontokon keresztül küldi az információkat, éppen ezért nagyon fontos, hogy az összes végpont megfelelően működjön. Ezek az adat csomagok JSON típusúak és elengedhetetlen, hogy az egyes végpontok helyes formában kapják meg, illetve továbbítsak ezek az adat csomagokat, azért, hogy 200, illetve 201-es sikeres kóddokkal térjenek vissza a kéréseink.

200: Azt jelzi, hogy a kért művelet sikeresen végrehajtódott.

201: Azt jelzi, hogy a kérés sikeres volt, és a kívánt erőforrás létrehozásra került

Felhasználói végpontok

Account végpontok:

1. /Register:

Amikor regisztrálunk ezen a végponton fognak keresztül menni az adatok, amiket megadunk. Ha 200-as kóddal tér vissza sikeresen megtörtént a regisztráció és legenerált a program nekünk egy belépési tokenet, amivel bejelentkezhethetünk. Az alábbi képeken látható a JSON formájú adat, amit megkap és a sikeres visszatérési kód. (A képeken a tokeneknek csak egy része látható), POST típusú.

18. ábra /register curl

19. ábra s register sikeres 200 -as válasz

20. ábra login curl

21. ábra login sikeres 200-as válasz

23. ábra refresh-token curl

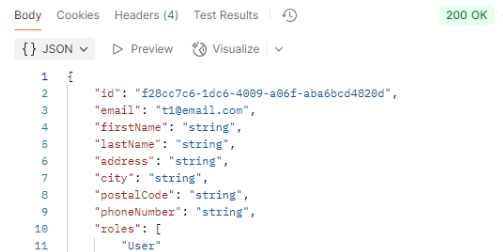
22. ábra refresh-token sikeres 200-as válasz

4./get-me:

A get-me végpont a belépett felhasználónak az adatait kéri le. Ezt a végpontot hívjuk meg amikor a weboldalon a felhasználó rákattint a fiókjára és megjelenik előtte az adatai. GET típusú.

```
curl -X 'GET' \
  'https://localhost:7059/api/account/get-me' \
  -H 'accept: text/plain' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsI
```

24. ábra get-me curl



```
Body Cookies Headers (4) Test Results 200 OK
JSON Preview Visualize
1 {
2   "id": "f28cc7c6-1dc6-4089-a06f-aba6bcd4820d",
3   "email": "t1@email.com",
4   "firstName": "string",
5   "lastName": "string",
6   "address": "string",
7   "city": "string",
8   "postalCode": "string",
9   "phoneNumber": "string",
10  "roles": [
11    "User"
```

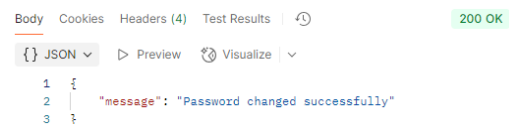
25. ábra get-me sikeres 200-as válasz

5./change-password:

A change password végpont ahogy a neve is jelzi, a jelszó változtatásáért felelős. Bekéri a régi jelszavát és az újat is. ha a régi nem egyezik az újjal akkor nem fogja megváltoztatni. Ha az új jelszó nem felel meg a követelményeknek ugyancsak nem fogja megváltoztatni és tudat a hibáról. PUT típusú.

```
curl -X 'PUT' \
  'https://localhost:7059/api/account/change-password' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6I
```

27. ábra change-password curl



```
Body Cookies Headers (4) Test Results 200 OK
JSON Preview Visualize
1 {
2   "message": "Password changed successfully"
3 }
```

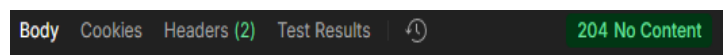
26. ábra change-password 200-as sikeres válasz

6./ {id}/edit-user:

Ezen a végponton a felhasználók a fiókjuknak az adatait változtathatják egy PUT típusú kéréssel. Az összes regisztrációnál megadott adatnak a változtatására is van lehetőség. A nevében lévő id arra utal, hogy minden felhasználó csakis a saját fiókjának adatait változtathatja meg, ez a lekérés csakis egy fiók adataival tér vissza.

```
curl -X 'PUT' \
  'https://localhost:7059/api/account/
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6I
```

28. Ábra id}/edit-user curl



```
Body Cookies Headers (2) Test Results 204 No Content
```

29. Ábra id}/edit-user 204-es sikeres válasz

Order végpontok

1. Orders: Az orders végponton keresztül történik a termékek megrendelése. Megkapja a rendelt termékek id-ját és számát, majd visszaadja a címet, a státuszát, a dátumát és a teljes fizetendő összeget többek között. POST típusú.

```
curl -X 'POST' \
  'https://localhost:7059/api/Orders' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6YW50IjoiOiI' \
  -H 'Content-Type: application/json' \
  -d '{
    "deliveryAddress": "string",
    "cart": {
      "items": [
        {
          "productId": 1,
          "quantity": 2
        }
      ]
    }
  }'
```

30. ábra Orders curl

```
Body Cookies Headers (5) Test Results 201 Created 83 ms
JSON Preview Visualize
1 {
2   "id": 15,
3   "userFullName": "string string",
4   "orderDate": "2025-04-24T11:04:38.4283993Z",
5   "totalAmount": 6598.0,
6   "status": "Pending",
7   "deliveryAddress": "string, string, string",
8   "orderItems": [
9     {
10      "id": 19,
11      "productId": 1,
```

31. ábra Orders 201-es sikeres válasz

2./my-orders

A my-orders végponton a felhasználó a saját, eddigi rendeléseit tudja visszakérni. GET típusú.

```
curl -X 'GET' \
  'https://localhost:7059/api/Orders/my-orders' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6YW50IjoiOiI'
```

32. ábra my-orders curl

```
Body Cookies Headers (4) Test Results 200 OK 17 ms
JSON Preview Visualize
1 [
2   {
3     "id": 15,
4     "userFullName": "string string",
5     "orderDate": "2025-04-24T11:04:38.4283993Z",
6     "totalAmount": 6598.0,
7     "status": "Pending",
8     "deliveryAddress": "string, string, string",
9     "orderItems": [
10      {
11        "id": 19,
12        "productId": 1,
```

33. ábra my-orders 200-as sikeres válasz

- 3./my-orders/{i} A my orders {id} az adott felhasználó eddigi rendelései közt lévő egy darab rendelést adja vissza a rendelés id-ja alapján. GET típusú.

```
curl -X 'GET' \
  'https://localhost:7059/api/Orders/14' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6YW50IjoiOiI'
```

34. ábra my-orders/{i} curl

```
Body Cookies Headers (4) Test Results 200 OK 14 ms
JSON Preview Visualize
1 {
2   "id": 14,
3   "userFullName": "string string",
4   "orderDate": "2025-04-24T11:04:15.3209528Z",
5   "totalAmount": 6598.0,
6   "status": "Pending",
7   "deliveryAddress": "string, string, string",
8   "orderItems": [
9     {
10      "id": 18,
11      "productId": 1,
```

35. ábra my-orders/{i} 200-as sikeres válasz

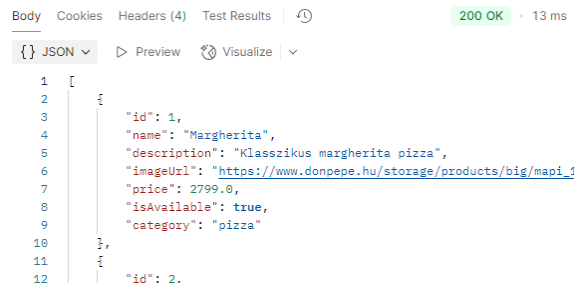
Products végpontok:

1. /Products

A products végpont a termékek meghívásáért felel. GET típusú.

```
curl -X 'GET' \
  'https://localhost:7059/api/Products' \
  -H 'accept: text/plain' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpzZW50L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9'
```

36. ábra Products curl



```
1 [
2   {
3     "id": 1,
4     "name": "Margherita",
5     "description": "Klasszikus margherita pizza",
6     "imageUrl": "https://www.donpepe.hu/storage/products/big/mapi_1.jpg",
7     "price": 2799.0,
8     "isAvailable": true,
9     "category": "pizza"
10  },
11  {
12    "id": 2.
```

37. ábra Products 200-as sikeres válasz

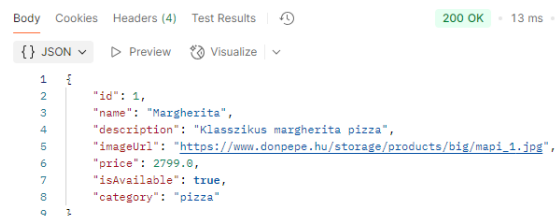
2. /Products/{id}

Ez a végpont az egyes termékek meghívásáért felel a saját id-juk alapján.

GET típusú.

```
curl -X 'GET' \
  'https://localhost:7059/api/Products/1' \
  -H 'accept: text/plain' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpzZW50L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9'
```

38. ábra Products/{id} curl



```
1 {
2   "id": 1,
3   "name": "Margherita",
4   "description": "Klasszikus margherita pizza",
5   "imageUrl": "https://www.donpepe.hu/storage/products/big/mapi_1.jpg",
6   "price": 2799.0,
7   "isAvailable": true,
8   "category": "pizza"
9 }
```

39. ábra Products/{id} 200-as sikeres válasz

Admin végpontok

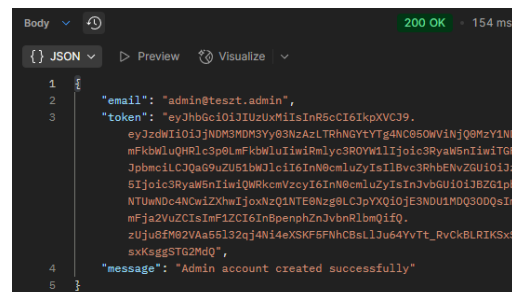
1. Admin/create-admin

Ezen a végponton keresztül tudjuk az admin fiókokat létrehozni. Ha a cég szeretne új admin fiókot létrehozni akkor ahhoz minket kell megkeresniük mert biztonsági okokból csakis a kód készítői tudnak csak új admin jogosultságú fiókot létrehozni.

POST típusú.

```
curl -X 'POST' \
  'https://localhost:7059/api/admin/create-admin' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpzZW50L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9' \
  -d '{
    "email": "admin@teszt2.admin",
    "password": "mdINSIJAN3848732dwm28euUS9DDISNF32.",
    "phoneNumber": "string",
    "firstName": "string",
    "lastName": "string",
    "address": "string",
    "city": "string",
    "postalCode": "string"
  }'
```

41. ábra create-admin curl



```
1 {
2   "email": "admin@teszt2.admin",
3   "token": "eyJhbGciOiJIUzUxMiIsInR5cCI6IkpzZW50L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJ1b290L3Nja2VudCIsInR0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9",
4   "message": "Admin account created successfully"
5 }
```

40. ábracreate-admin 200-as sikeres válasz

2. /All-orders

Ez a végpont kéri le az összes eddigi rendelést, a rendelések adataival együtt. A weboldalon az admin dashboard felületen, az all orders fül tartalmát adja. GET típusú

```
curl -X 'GET' \
  'https://localhost:7059/api/orders/all-orders' \
  -H 'accept: text/plain' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6I
```

43. ábra all-orders curl

```
Body 200 OK 34 ms
{} JSON Preview Visualize
1 [
2   {
3     "id": 3,
4     "orderDate": "2025-04-24T13:58:44.2307609",
5     "totalAmount": 2799.0,
6     "status": "Completed",
7     "deliveryAddress": "string, string, string",
8     "fullUserName": "string string",
9     "userEmail": "admin@admin.admin",
10    "userId": "930439c1-8caa-4160-aace-08c13ab7cd15",
11    "orderItems": [
12      {
13        "id": 3,
14        "productId": 1,
15        "product": {
16          "id": 1,
17          "name": "Margherita",
18          "description": "Klasszikus margherita pizza",
19          "imageUrl": "https://example.com/margherita.jpg",
20          "price": 2799.0,
21          "isAvailable": true,
22          "category": "pizza"
23        },
24        "quantity": 1,
25        "price": 2799.0,
26        "totalPrice": 2799.0
27      }
28    ]
29  }
30 ]
```

42. ábra all-orders 200-as sikeres válasz

3. Orders/Status/{i}

Ezen a végponton az adminoknak lehetőségük van az egyes rendelések státuszát állítani mely lehet, pending, completed, és cancelled, PATCH típusú lekérés.

```
curl -X 'PATCH' \
  'https://localhost:7059/api/orders/3/status' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6I
  -H 'Content-Type: application/json' \
  -d '{
    "status": "Completed"
  }'
```

45. ábra order-status curl

```
Body 200 OK 8 ms
{} JSON Preview Visualize
1 {
2   "id": 3,
3   "userFullName": " ",
4   "orderDate": "2025-04-24T13:58:44.2307609",
5   "totalAmount": 2799.0,
6   "status": "Completed",
7   "deliveryAddress": "string, string, string",
8   "orderItems": []
9 }
```

44. ábra order-status 200-as sikeres válasz

4. /Products

A végpont megadja a lehetőséget az adminoknak az új termékek felvételére, tekintve, hogy helyesen adják meg a szükséges adatokat, PUT típusú.

```
curl -X 'POST' \
  'https://localhost:7059/api/products' \
  -H 'accept: text/plain' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6I
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Teszt",
    "description": "Teszt",
    "imageUrl": "Teszt.jpg",
    "price": 1111,
    "category": "pizza",
    "isAvailable": true
  }'
```

46. ábra products curl

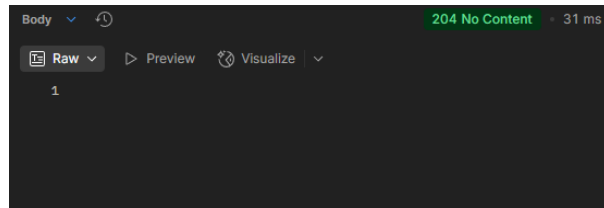
```
Body 201 Created 46 ms
{} JSON Preview Visualize
1 {
2   "id": 5,
3   "name": "Teszt",
4   "description": "Teszt",
5   "imageUrl": "Teszt.jpg",
6   "price": 1111,
7   "isAvailable": true,
8   "category": "pizza"
9 }
```

47. ábra products 201-as sikeres válasz

5. /Products/{id}

Itt az egyes termékek adatait változtathatják meg az adminok, PUT típusú.

```
curl -X 'PUT' \
  'https://localhost:7059/api/Products/5' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXLTJ5In0:eyJ1IjoiYm9keSIsInR5cCI6IkpXLTJ5In0' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 5,
    "name": "Pizza",
    "description": "Pizza",
    "imageUrl": "Pizza",
    "price": 3333,
    "category": "pizza"
  }'
```

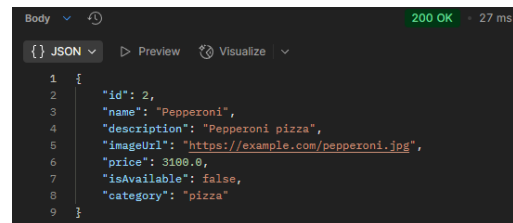


49. ábra *Products/{id} curl*

48. ábra *Products/{id}* 200-as sikeres válasz

6./Products/{id}/availability

Itt a termékeknek csakis az elérhetőségét lehet állítani az adminoknak. PUT típusú

[illegible]

50. ábra *Products/{id}/availability* curl

51. ábra *Products/{id}/availability* 200-as sikeres válasz

3. Extrém Teszteset: A felhasználó valóságghűtlen hosszú jelszavat szeretne beállítani.

Elvárás: A program ne engedje a természetellenesen hosszú jelszavak felvételét.

Művelet: felhasználó egy 1000 karakter hosszúságú jelszavát szeretne beállítani a fiókjához.

Várt eredmény: A program egyértelműen és pontosan jelezze a jelszó érvénytelenséget és ne engedje a regisztrációt amíg az nincs megváltoztatva egy érvényesre.

Valós eredmény: A program piros univerzális stop szimbólummal jelzi a jelszó érvénytelenséget a mező inaktív állapotában. (A felhasználó ki kattint a mezőből).

Password 



52. ábra Hibás jelszó mező

Aktív állapotban (a felhasználó bele kattint a mezőbe) kiírja a szükséges követelményeket.

12 to 24 characters. Must include uppercase and lowercase letters, a number and a special character. Allowed special characters: ! @ # \$ %

53. ábra Jelszó követelmény üzenet

IV. Felhasználói dokumentáció

1. A program általános specifikációja, célja

Termékünk egy felhasználó barát és intuitív környezetet alkot a vásárlóknak, akik gondmentesen és biztonságosan nézelődhetnek és vásárolhatnak az otthonuk vagy mobileszközük nyújtotta kényelemből amiért több hasznos funkciót is implementáltunk.

Regisztr & Login: A felhasználóknak biztonsága érdekében, amikor először látogatják az oldalunkat meg lesznek kérve, hogy regisztráljanak vagy ha már rendelkeznek fiókkal, jelentkezzenek be. A regisztrációnál megadott adatokat később a rendelési folyamat egyszerűsítésére fogjuk alkalmazni, például email-t tudunk küldeni a felhasználóknak a rendelésük adatairól és sikerességéről.

Kosár: Az oldalunkon a vásárlás és nézelődés folyamatát virtuális kosarak létrehozásával könnyítettük meg. A folyamat bármely pontján a látogatóknak lehetőségük van megnézni, hogy eddig milyen termékeket terveznek megvenni és mennyit, illetve mennyibe kerülnek az egyes tételek és összesen.

Rendelés: Amikor a vásárló úgy érzi minden terméket, amit megszeretne vásárolni a kosárba rakta akkor a „Megrendelés” gombra kattintva elindíthatja a rendelés folyamatát. A gombra kattintás után az oldal adni fog egy rövid összefoglalást a megrendelni kívánt termékekről, hogy a vásárló még egyszer le ellenőrizhesse, hogy biztosan azokat a termékekért fizet, amelyeket megakar venni. Ezután a szállítási adatokat kell majd a vásárlóknak megadnia, a weboldal felfogja ajánlani a regisztrációnál megadott adatokat, de a felhasználó adhat meg más adatokat tetszés szerint. Termékünk jelenlegi állapotában csak készpénzes fizetésre van lehetőség, de a jövőben tervezzük az online fizetésre való lehetőség bevezetését. Ha idáig megvagyunk egy végső megerősítést kell adni a rendelésről és a rendszerünk el is kezdi a rendelés feldolgozását, amiről egy megerősítő emailt is kifogunk küldeni.

Kinézet és Design: A weboldalunk egy letisztult és egy esztétikailag szép kinézetet biztosít az egyszerű használat céljából. A termékekhez illusztrációként képeket társítottunk, amelyek vizuálisan segítik a felhasználókat a választásban. A képek illusztrációs célt szolgálnak, és eltérhetnek a valóságos megjelenéstől.

Admin Dashboard: Az admin jogkörrel rendelkező fiókoknak van egy Admin Dashboard nevezetű fül, ahol megtudják tekinteni az eddig leadott rendeléseket, új termékeket adhatnak hozzá vagy esetleg már meglévő termékeknek az információját frissíthetik.

My Orders: Az account oldalon a felhasználóknak lehetőségük van a saját eddig leadott rendelésük megtekintésére. Láthatják, hogy a rendeléseket hova szállították ki, mi volt a tartalma, mikor adták le és mekkora összegben.

Change Password & User Info: A felhasználók bármikor megváltoztathatják a jelszavukat és a fiókjuk információit.

Visszaigazoló email: Sikeres rendelés után a weboldal küldeni fog a felhasználóknak egy visszaigazoló emailt a rendeléseiknek az adataival.

2. Rendszer követelmények

Általános követelmények:

- Operációs rendszer: Windows 10+, macOS 10.14+ vagy modern Linux disztribúció (pl. Ubuntu 20.04+)
- Internetkapcsolat a függőségek letöltéséhez

Backend (C# / .NET)

- .NET SDK 8.0 vagy újabb
- Adatbázis: SQLite)
- Parancssoros eszközök (Terminál, PowerShell vagy CMD)
- Backend helyi címe szabad a `http://localhost:7059`-ös porton

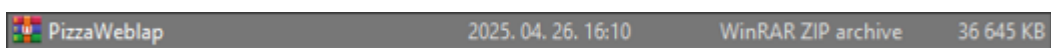
Frontend (React + Vite)

- Node.js 16.0 vagy újabb
- npm (a Node.js része) vagy Yarn
- Modern böngésző (pl. Chrome, Firefox, Edge)
- Frontend helyi címe szabad legyen a `http://localhost:3000`-es porton.

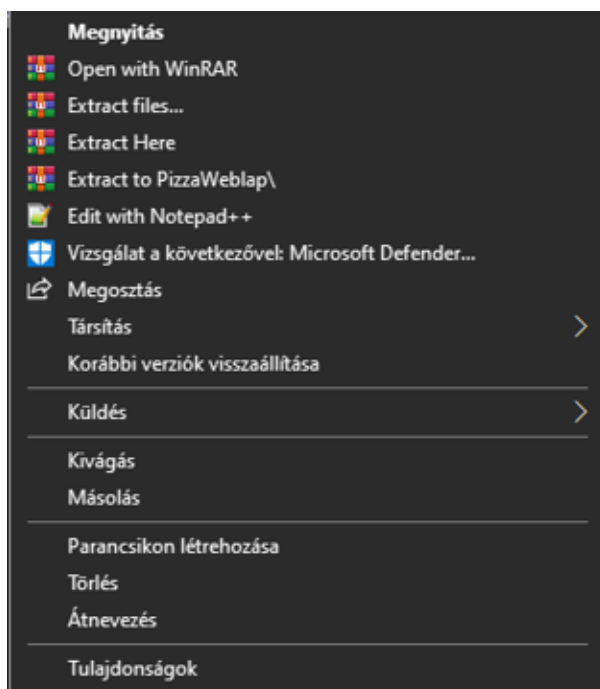
3. A program telepítése

Ha rendelkezünk a szükséges tömörített fájl (.zip kiterjesztésű) helyezzük el a kívánt mappába, fontos, hogy legalább 1gb hellyel rendelkezünk a kiválasztott lemezen, de ideális a több. Fontos, hogy rendelkezünk egy kicsomagoló programmal, ami nélkülözhetetlen a program futtatásához.

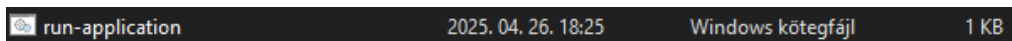
Ha felmásoltuk a .zip fájlt a kívánt lemezre ki kell bontanunk. Ezt úgy tehetjük meg hogy rávisszük a kurzort a fájlra és egyszer rákattintunk egy bal klikkel miután kihalványul a háttere, ez jelzi, hogy kiválasztottuk a fájlt. Majd egyszer nyomjunk egy jobb kattintást, ami kifog adni egy sor opciót. Itt tetszés szerint kiválaszthatjuk, hogy egyszerűen ide csomagolja ki (extract here), hozzon létre egy PizzaWeblap nevű mappát (extract to pizza backend) vagy, egy teljesen másik helyen lévő mappába (extract files...), csomagolja ki. A program futására nincsen kihatással a fájl helye amíg elegendő tárhely áll rendelkezésre így annak elhelyezése teljesen szubjektív és a vásárló döntése. Ezután már láthatónak kell lenni a mappának. Kattintsunk bele a PizzaWeblap nevű mappába.



54. ábra Kijelölt .zip fájl

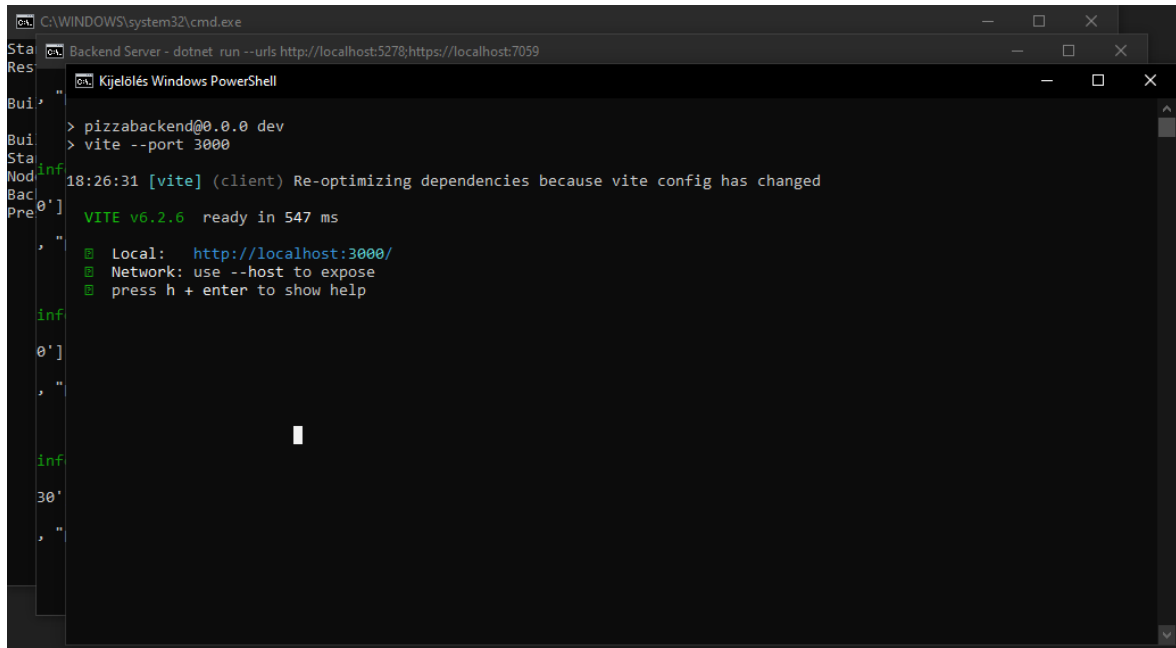


55. ábra Kicsomagolási opciók



56. Ábra run application fájl

Itt 3 fájl és 2 mappa fog minket fogadni, nekünk csak a run-application nevezetű .bat típusú fájl lesz fontos. Egy gyors dupla kattintással indítsuk is el. Ez a fájl megnyit három PowerShell ablakot majd letölti a szükséges csomagokat és elkezdi futtatni a backend szervert, a frontend applikációnkat és automatikusan megnyitja a weboldalunkat és már használhatjuk is a weboldalt.



```
C:\WINDOWS\system32\cmd.exe
Backend Server - dotnet run --urls http://localhost:5278;https://localhost:7059

Kijelölés Windows PowerShell

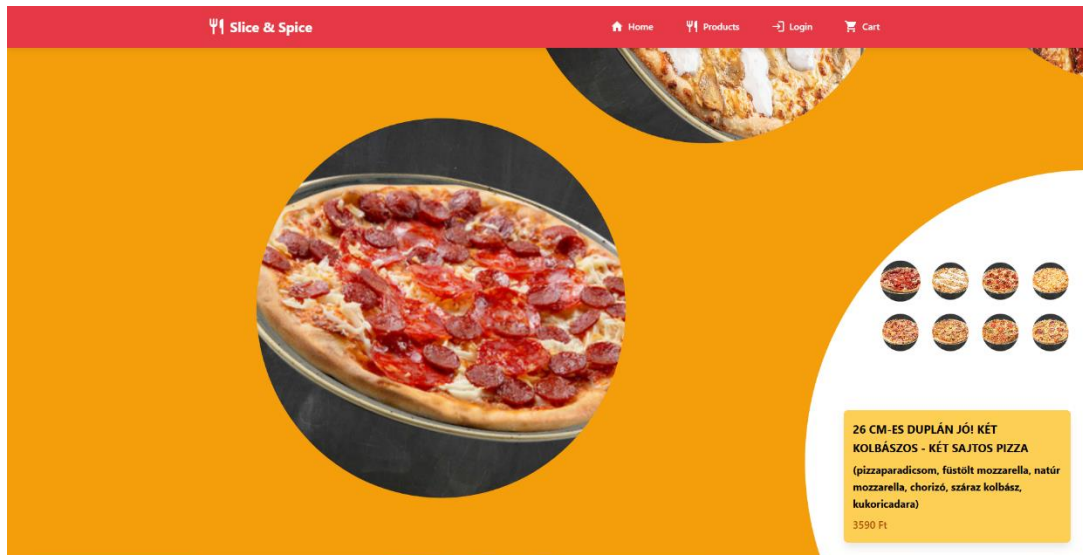
> pizzabackend@0.0.0 dev
> vite --port 3000

18:26:31 [vite] (client) Re-optimizing dependencies because vite config has changed
VITE v6.2.6 ready in 547 ms
  Local:   http://localhost:3000/
  Network: use --host to expose
  press h + enter to show help
```

57. Ábra tun-application futtatása

4. A program használatának részletes leírása

Amikor felnyitjuk az oldalt egy forgó tálcához hasonló animációs főoldal fogad minket, ahonnan a login földre navigáljunk és jelentkezünk be, vagy hozzunk létre egy fiókot, ha még nem rendelkezünk eggyel. (A demonstráció során én egy admin fiókkal fogok dolgozni)



58. Ábra Főoldal

Welcome Back

Please sign in to continue

Email

Password

Sign In

Don't have an account? [Register now](#)

60. ábra Bejelentkezés

Username

Email Address

Password

First Name **Last Name**

Phone Number

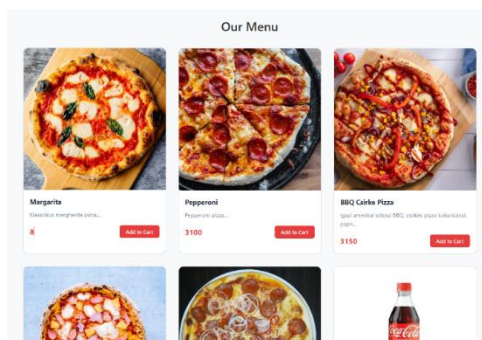
Address

City **Postal Code**

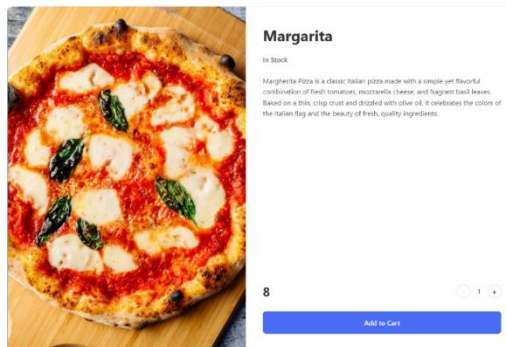
Register

59. ábra Regisztráció

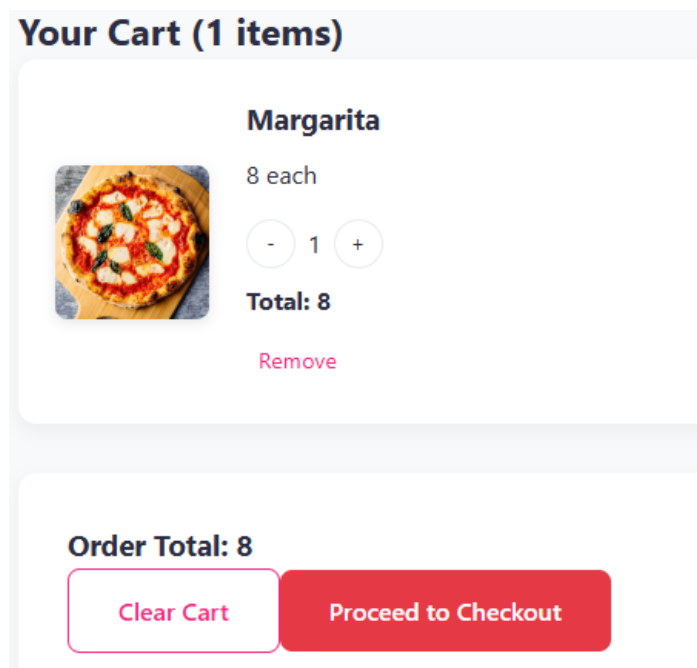
Ez után neki láthatunk a termékek közötti böngészésnek kedvünk szerint, ha rákattintunk az egyes termékekre több információt is kaphatunk róluk. Ha rákattintunk, hogy Add to Cart a jobb felső sarokban a kosár ikonján megjeleni egy számláló, ami mutatja, hogy mennyi terméket raktunk bele. Ha a vásárlás bármelyik pontján érdekel, hogy mely termékeket raktuk a kosárba eddig, illetve mennyit kell majd fizetnünk az ikonra kattintva megtekinthetjük.



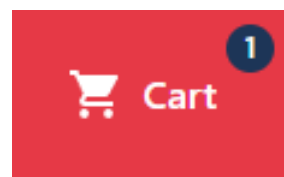
64. ábra Products oldal



63. ábra Product oldal



62. ábra Kosár oldal



61. ábra Termék számláló

Checkout

Shipping Information

Full Name

Address

City

ZIP Code

☐ Shipping to a different address?

Payment Method

☐ Credit Card

☒ Cash

Place Order - 0

Your Order		
	Margherita 2798 × 1 Ft	2798 Ft
	Pepperoni 3100 × 1 Ft	3100 Ft
	PEPSI COLA 699 × 1 Ft	699 Ft
Subtotal		6597 Ft
Shipping		Free
Total		6597 Ft

Thank You for Your Order

	Margherita Quantity: 1	2798 Ft
	Pepperoni Quantity: 1	3100 Ft
	PEPSI COLA Quantity: 1	699 Ft
Order Total		6597,00 Ft

33

Order Confirmed!

Thank you for your purchase

Order Details

Order Number:
#18

Date:
4/25/2025

Margherita

Qty: 1

2798 Ft

Pepperoni

Qty: 1

3100 Ft

PEPSI COLA

Qty: 1

699 Ft

Subtotal

6597 Ft

Delivery Fee

Free

Total

6597 Ft

70. Ábra Sikeres rendelés üzenet

7030, Paks, Vihar utca 44

Payment Method: Cash on Delivery

Continue Shopping

68. Ábra Sikeres rendelés címe

Az oldalon továbbá lehetőség van a fiók adatainak megváltoztatására, a rendelési előzmények megtekintésére és a jelszó megváltoztatására is, csak írjuk át a kívánt adatokat és mentjük el őket.

My Account

User info

My Orders

Change Password

User Information

Email

admin@admin.com

First Name

Kovács

Last Name

Péter

Address

Lázár utca 34

Phone

+36 20 435 5643

City

Paks

ZIP Code

7030

Save Changes

73. ábra Fiók adatok

Change Password

Current Password

New Password

Confirm New Password

Update Password

72. ábra Jelszó változtatás

Order History

Order #8

Pending

Margarita

Qty: 1

2799

Date: 4/18/2025

Delivery Address: string, string, string

Total: 2799

71. ábra Rendelési előzmény

34

Az admin dashboard egy rejtett fül a weboldalon melyen csak az admin jogkörrel ellátott tokennel biztosított felhasználók láthatnak és használhatnak. Ezen a fülön olyan műveletek elérhetőek, mint például új termékek hozzáadása, a már meglévő termékek adatainak megváltoztatása, az összes eddigi leadott rendelés előzményeinek megtekintése, illetve ezeknek a rendeléseknek a státuszának állítása.

Manage Products

Add Product

ID	Image	Name	Description	Price	Status	Actions
1		Margarita	Margherita Pizza is a classic Italian pizza made with a simple yet flavorful combination of fresh tomatoes, mozzarella cheese, and fragrant basil leaves. Baked on a thin, crisp crust and drizzled with olive oil, it celebrates the colors of the Italian flag and the beauty of fresh, quality ingredients.	\$8.00	☒	Edit
2		Pepperoni	A true classic loved by all — a rich tomato sauce base topped with generous slices of spicy pepperoni and a blanket of gooey mozzarella cheese, baked to golden perfection.	\$8.00	☒	Edit
3		BBQ Csirke Pizza	Smoky and savory! This pizza features tender grilled chicken, smothered in a sweet and tangy barbecue sauce, complemented by a blend of melted cheeses atop a crispy crust.	\$8.00	☒	Edit
4		Hawai Pizza	A tropical twist on tradition — this pizza combines juicy pineapple pieces with slices of smoky ham over a tomato sauce and cheese base, creating a perfect balance of sweet and savory flavors.	\$8.00	☒	Edit
5		Magyaros Pizza	Inspired by bold Hungarian flavors, the Magyaros pizza is loaded with spicy sausage, smoky bacon, fresh peppers, and onions, all layered over a rich tomato sauce and melted cheese.	\$8.00	☒	Edit
6		Coca Cola	The ultimate classic soft drink — Coca-Cola's iconic blend of sweetness and refreshing carbonation makes it the perfect pairing for any meal.	\$2.00	☒	Edit
7		Tonic	Light, crisp, and effervescent, this tonic water offers a refreshing bite, enjoyed on its own or as the perfect mixer for your favorite spirits.	\$2.00	☒	Edit

77. ábra Admin Products

Edit Product

Name

Margherita

Description

Klasszikus margherita pizza

Price

2799

Category

Pizza

Image URL

https://www.donpepe.hu/storage/products/big/mapi_1.jpg

☒ Available

Cancel

Save

75. ábra Admin product változtatás

Add New Product

Name

Description

Price

0

Category

Select Category

Image URL

☒ Available

Cancel

Save

74. ábra Új termék felvétele

Manage Orders

Order ID	Customer	Date	Total	Status	Actions
14	Kovács Péter	4/24/2025	2799 HUF	Pending	View

76. ábra Admin rendelés

V. Összegzés

Termékünk jelenleg egy kezdetleges fázisban van viszont számtalan lehetőséget biztosít az időbeli fejlesztésre többek között:

Discount: Ha a tulajdonsok szeretnék majd kuponokat, kedvezményeket biztosítani a vásárlóik számára, meg lesz rá a lehetőségük a discount táblán keresztül, amit a vásárlás előtti áttekintő oldal és a kosár is képes lesz megjeleníteni.

Review: A review funkció megadná a lehetőséget a vásárlóknak a vissza jelzés nyújtására. Nem csak egy 5/x csillag rendszer formájában, hanem személyes vélemények komment formájú rögzítéseként is.

Online fizetés: Programunk jelenleg csak készpénben való fizetés feldolgozására képes, de a payment táblán keresztül nyitva áll a kapu a különböző online fizetések hozzáadásának lehetőségére is mint például a Paypal, Smart pay, vagy esetleg szépkártya.

Szerver Kosár: Termékünk jelenleg a kliens oldali local Storage-ben tárolja el a kosarakat és annak tartalmait, természetesen, ha kilépünk az oldalról és visszalépünk akkor is megmarad viszont, ha egyszer már leadtuk a rendelést és inaktívra lett állítva a kosár akkor már nem tárolja el tovább, amit a szerver oldalon, a backendben eltárolt kosár elmentése megoldana a jövőben.

VI. Ábra jegyzék

2. ábra User Szekció	9
1. Ábra User szekció	9
3. ábra Order szekció	10
4. ábra Product szekció	11
5. ábra In Development szekció.....	12
6. ábra Teljes adatbázis modell.....	13
7. ábra Regisztrációs adatok előkészítése	14
8. ábra Regisztrációs adatok elküldése a végpontra	14
9. ábra Bejelentkezési adatok elküldése.....	15
10. ábra A bejelentkezés utáni műveletek.....	15

11. ábra Feltöltjük a setAuthot a kinyert adatokkal	15
12. ábra A kötelező header hozzáadása az összes kéréshez	16
13. Ábra useRefresh Struktogram.....	18
14. ábra Sikertelen jelszó változtatás üzenet	19
17. ábra Sikeres jelszó változtatás üzenet	19
15. ábra Nem egyező jelszavak hiba üzenet.....	19
16. ábra Jelszó követelmények üzenet	19
18. ábra /register curl	20
19. ábra s register sikeres 200 -as válasz.....	20
20. ábra login curl	20
21. ábra login sikeres 200-as válasz.....	20
22. ábra refresh-token sikeres 200-as válasz	20
23. ábra refresh-token curl.....	20
24. ábra get-me curl	21
25. ábra get-me sikeres 200-as válasz.....	21
26. ábra change-password 200-as sikeres válasz	21
27. ábra change-password curl	21
28. Ábra id}/edit-user curl.....	21
29. Ábra id}/edit-user 204-es sikeres válasz	21
30. ábra Orders curl.....	22
31. ábra Orders 201-es sikeres válasz	22
32. ábra my-orders curl	22
33. ábra my-orders 200-as sikeres válasz.....	22
34. ábra my-orders/{i} curl	22
35. ábra my-orders/{i} 200-as sikeres válasz	22
36. ábra Products curl.....	23

37. ábra Products 200-as sikeres válasz	23
38. ábra Products/{id} curl.....	23
39. ábra Products/{id} 200-as sikeres válasz	23
40. ábra create-admin 200-as sikeres válasz	23
41. ábra create-admin curl.....	23
42. ábra all-orders 200-as sikeres válasz.....	24
43. ábra all-orders curl	24
44. ábra order-status 200-as sikeres válasz	24
45. ábra order-status curl.....	24
46. ábra products curl.....	24
47. ábra products 201-as sikeres válasz	24
48. ábra Products/{id} 200-as sikeres válasz	25
49. ábra Products/{id} curl.....	25
50. ábra Products/{id}/availability curl	25
51. ábra Products/{id}/availability 200-as sikeres válasz	25
52. ábra Hibás jelszó mező.....	26
53. ábra Jelszó követelmény üzenet.....	26
54. ábra Kijelölt .zip fájl.....	29
55. ábra Kicsomagolási opciók.....	29
56. Ábra run application fájl.....	29
57. Ábra tun-application futtatása.....	30
58. Ábra Főoldal	31
59. ábra Regisztráció.....	31
60. ábra Bejelentkezés.....	31
62. ábra Termék számláló	32
61. ábra Kosár oldal	32

63. ábra Product oldal	32
64. ábra Products oldal.....	32
65. ábra Checkout	33
66. ábra Kiszállítási ablak	33
67. Ábra Visszaigazoló email kinézete	33
70. Ábra Sikeres rendelés címe	34
68. Ábra Rendszer üzenet az email kiküldéséről.....	34
69. Ábra Sikeres rendelés üzenet.....	34
71. ábra Rendelési előzmény	34
72. ábra Jelszó változtatás	34
73. ábra Fiók adatok.....	34
74. ábra Új termék felvétele	35
75. ábra Admin product változtatás	35
76. ábra Admin rendelés.....	35
77. ábra Admin Products.....	35

VII. Irodalomjegyzés, hivatkozási jegyzék

Szoftverek:

- <https://visualstudio.microsoft.com>
- <https://code.visualstudio.com>
- <https://www.npmjs.com>
- <https://nodejs.org/en>

Programozási segítségék:

- <https://www.w3schools.com/react/>
- <https://react.dev>
- <https://learn.microsoft.com/en-us/dotnet/csharp/>

Packages:

- <https://www.npmjs.com/package/react>
- <https://www.npmjs.com/package/react-dom>
- <https://www.npmjs.com/package/react-router-dom>
- <https://www.npmjs.com/package/react-toastify>
- <https://www.npmjs.com/package/framer-motion>
- <https://www.npmjs.com/package/@mui/material>
- <https://www.npmjs.com/package/vite>

Fejlesztői eszközök:

- <https://www.npmjs.com/package/@vitejs/plugin-react>
- <https://www.npmjs.com/package/eslint-plugin-react-hooks>
- <https://www.npmjs.com/package/eslint-plugin-react-refresh>
- <https://www.npmjs.com/package/globals>
- <https://www.npmjs.com/package/@typescript-eslint/eslint-plugin>